

THE FLIGHT RECORDER

WAFT is a Python framework for directed evolution of self-modifying AI agents. Think of it as an operating system for AI agent research projects. Instead of just building agents that execute code, WAFT enables you to breed agents that can modify their own code, evolve through mutations, and be tested in fitness systems with complete lineage tracking for scientific research.

1. Methodology

The Flight Recorder is a rigorous telemetry system for generating phylogenetic trees of agent lineage. Every evolutionary action is recorded with complete context including genome ID, parent ID, generation number, event type, payload with complete context, and fitness metrics. This enables reconstruction of complete family trees for scientific publication, allowing phylogenetic analysis, mutation impact measurement, fitness landscape mapping, and convergence analysis. The Evolutionary Document Creator System transforms any conversation into exactly 2-page printable documents using evolved styling. The system treats both ideas and styling as genetic material with genome IDs, scientific names, lineage tracking, and fitness evaluation. Natural selection optimizes document designs over generations. WAFT is ambient, working quietly in the background without getting in your way. It is self-modifying, allowing projects to evolve their own structure over time. It is a meta-framework that orchestrates existing tools rather than replacing them. Everything is file-based, making it git-friendly and portable. The system includes gamification with D&D-style progression, epistemic tracking to know what you know and don't know, and scientific observation with complete lineage tracking. Version 0.5.2 adds evolutionary document creation, component evolution, user feedback learning, and self-documentation. WAFT is built to produce data for a future book or paper on the Physics of Artificial Cognition. The system is designed to track complete evolutionary lineages as phylogenetic trees, measure fitness through rigorous testing in the Scint Gym, record all mutations with complete context in the Flight Recorder, and enable scientific analysis of agent evolution patterns. This makes WAFT not just a framework but a scientific instrument. The WAFT repository is available on GitHub for exploration and contribution. Comprehensive documentation covers the AI SDK vision, agent interface design, evolutionary architecture, and state of the art research. The system is MIT licensed and actively developed. You can start with the quick start guide, explore examples, read the documentation, and join the community to learn more about breeding AI agents. Don't just build agents. Breed them. WAFT transforms AI agents from passive assistants into active project participants that can improve themselves and the projects they work on. The ultimate goal is to observe a God-Head agent emerge from thousands of generations of directed mutation and selection. Component Evolution introduces a genetic ancestry system for page assembly. Components have traits like `min_pages`, height preferences, and section preferences. The `ComponentEvolutionEngine` tracks component lineage, measures fitness, and evolves better layouts through mutation and selection. Components learn from user feedback to improve over time. User Feedback Collection enables the system to learn from real usage. Every document generation collects metrics on readability, completeness, constraint satisfaction, and aesthetics. This feedback drives evolution, making each generation better than the last. The system adapts to user preferences and document types.

WAFT: The Evolutionary Code Laboratory

2026-01-11

ABSTRACT

The Physics is the Scint System, which acts as a fitness function through Reality Fracture Detection.

2. Architecture

3. Conclusion

Self-Documentation System automatically generates documentation about the system itself. Components document their own behavior, evolution history, and fitness metrics. The system creates phylogenetic trees showing how components evolved, making the codebase self-explanatory and scientifically trackable. Integration with TheFoundation connects WAFT to a broader ecosystem of AI research tools. This enables cross-system learning, shared fitness functions, and collaborative evolution across multiple projects. TheFoundation provides shared infrastructure for scientific AI research. WAFT is scientific because it produces rigorous data for research publication on the physics of artificial cognition. It is evolutionary because agents evolve through genetic improvement, not just execution. It is observable because every action is recorded in the Flight Recorder for analysis. It is directed because evolution is guided by fitness functions, not random mutation. It is self-improving because systems learn from feedback and evolve better designs. WAFT provides project scaffolding through a unified CLI interface. You run one command to create a fully configured project with best practices built in. The system uses uv for fast Python package management, creates a `_pyrite` memory structure for organizing project knowledge, includes CI/CD pipelines ready to go, and provides optional AI agent templates. Everything is file-based with no database, no server, just plain text files that work with git. Install WAFT using uv tool `install waft`. Create a new evolutionary laboratory with `waft new my_laboratory`. Verify the substrate with `waft verify`. The system sets up everything you need including project structure, dependencies, CI/CD, and documentation templates. You can then spawn variants with mutations, evaluate fitness in the Gym, and evolve into the fittest variant. A WAFT laboratory includes `pyproject.toml` for uv project configuration, `uv.lock` for locked dependencies, a `_pyrite` directory for the memory system with `active`, `backlog`, and `standards` folders, GitHub Actions workflows for CI/CD pipelines, a Justfile for task running, and source code organized in a standard Python project structure. Everything is designed to be file-based and git-friendly. The `waft new` command creates a new evolutionary laboratory with all necessary structure. The `waft verify` command verifies the project structure is correct. The `waft evolve` command runs the evolutionary cycle for a target agent, spawning variants, evaluating fitness, and selecting the fittest. The `waft sync` command syncs project dependencies. The `waft add` command adds dependencies to the project. The `waft info` command shows information about the WAFT project. The `waft serve` command starts a web dashboard for visualization. WAFT doesn't lock you in. It's all file-based with no database to manage. Everything is plain text that works with git out of the box. You can modify anything because it's your project, and WAFT just set it up. The system is designed to be ambient, setting things up and getting out of your way so you can focus on building agents rather than configuring infrastructure. Version 0.5.2 extends this philosophy to document creation, making knowledge crystallized and printable.